Exception Handling

Thursday, October 9, 2025 4:41 PM

Exception Handling :

Exception handling refers to the process of responding to **unauthorized or unexpected events** that occur during the execution of a program, which may **stop the normal flow** of execution. These events, other than syntax errors, are called **exceptions**.

Example:

```
n1 = int(input('Enter the number1: '))
n2 = int(input('Enter the number2: '))
res = n1 / n2
print(res)
```

If the user enters 0 for n2, the program will throw a ZeroDivisionError and stop.

Q. What is the purpose of Exception Handling?

'''The purpose of exception handling is to ensure that the program **runs smoothly without interruptions**, even when unexpected errors occur during execution. It allows the program to **handle errors gracefully** instead of crashing.'''

Q. How can we handle exceptions in Python?

In Python, we primarily use **two keywords** to handle exceptions:

1. try – Contains the block of code that might raise an exception.
2. except – Contains the block of code to handle the exception if it occurs.

Exception handling can be done in **three ways**:

1. **Specific Exception Handling**
2. **Generic Exception Handling**
3. **Default Exception Handling**

1. Specific Exception Handling

- Used when we **know the type of error(s)** that might occur in the program.
- It allows us to handle a **specific exception** in a precise way, providing a tailored response.

Syntax:

```
try:
    <--->  ---program---
except ErrorName1:#ErrorName Should be Standard
   <--->  ------Soultion----
except ErrorName2:
    <--->  ------Soultion----
 except ErrorName n:
     <--->  ------Soultion n----
```

#Example:

```
def sam():
    try:
        n1=int(input('enter the number1:'))
        n2=int(input('enter the number2:'))
        res=n1/n2
        print(res)
    except ValueError:
        print('values for num1 and num2 should be integer')
        sam()
    except ZeroDivisionError:
        print('value for number2 should not be 0')
        sam()
sam()
```

Output:

```
enter the number1:10
enter the number2:0
value for number2 should not be 0
enter the number1:20
enter the number2:5
4.0
```

Generic Exception Handling:

Definition:

'''Generic Exception Handling is used when we **don't know what type of error** might occur in a program. Instead of handling a specific exception (like ValueError or ZeroDivisionError), we use the **generic Exception class** to handle any kind of runtime error.'''

Syntax1:

```
try:
    # program statements
except Exception:
    # solution or error handling
```

Syntax2:

```
try:
    # program statements
except Exception as variable_name:
    # print or handle the exception message
```

example for DefaultException:

```
def sample():
    try:
        n1 = int(input('Enter number 1: '))
        n2 = int(input('Enter number 2: '))
        res = n1 / n2
        print('Result:', res)
    except Exception as e:
        print("An error occurred:", e)
        # recursively calling the function to retry
        sample()

sample()
```

Explanation:

- Here, if any runtime error occurs (like division by zero, invalid input, etc.), the generic Exception block will handle it.
- The variable e stores the actual error message (for example, *ZeroDivisionError: division by zero*).
- Using Exception as e helps us know what kind of error occurred, even when we didn't specify it.

Default Exception Handling

Definition:

'''Default Exception Handling means the **built-in error handling** provided by Python when we **don't use any''**

try-except block.

If an exception occurs, Python automatically stops the program and prints the error message on the console.

However, we can also use a **general except block** (without specifying any exception type) to handle all errors, which is also sometimes referred to as *default handling* by programmers.

Syntax:

```
try:
    # program statements
except:
    # handle any type of error
```

Example:

```
try:
    a = int(input('Enter a number: '))
    print(a + 10)
except:
    print('Default Exception Handled')
```

Explanation:

- If the user enters a non-integer value, a ValueError occurs.
- Since we used except: (without specifying the type), it handles all types of exceptions.

Note:
In the above example, if you input a string instead of a number,
Python raises a ValueError, but since we used a general except: block, it gets handled by default.

Using All Three Types Together
Yes, we can use **specific, generic, and default exception handling** together,
but the order is important. Python checks in this order:

1. **Specific Exception** (e.g., ValueError)
2. **Generic Exception** (Exception as e)
3. **Default Exception** (except:)

Once an exception is handled, the remaining blocks are **skipped**.

Example:

```
try:
    a = int(input('Enter a number: '))
    print(a + 10)
except ValueError:
    print('ValueError: Please enter an integer value.')
except Exception as e:
    print('Generic Exception handled:', e)
except:
    print('Default Exception handled.')
```

Output Examples:

- If you enter 'abc', it triggers ValueError.
- If another unexpected error occurs, it goes to the **generic exception block**.
- The **default exception block** is the last fallback.

Differences:

Type	Used When	Syntax	Example
Specific Exception	You know the exact error type	except ValueError:	Handles only ValueError
Generic Exception	You don't know which error might occur	except Exception as e:	Handles all known error types
Default Exception	Used as a fallback for all errors	except:	Catches any remaining unhandled exceptions

Custom Exceptions
Definition:
'''Custom Exception Handling allows the programmer to **create and raise their own exceptions** according to specific requirements using the raise keyword.
It helps make error messages more meaningful and relevant to the program's context.'''

Syntax:raise ErrorName('Message')

Example:

```
noproducts = int(input('Enter the number of products: '))
if noproducts <= 0:
   raise ValueError('Out Of Stock')
else:
   print('Product is Available')
```

Explanation:
- If the user enters 0 or a negative number, the program manually raises a ValueError.
- This helps developers define errors according to **business logic** rather than relying only on Python's built-in ones.

User-Defined Exceptions
Definition:
'''User-Defined Exceptions are **custom error classes** created by the user.
They are used when built-in exception types are not suitable for a particular condition.
We create them using the class keyword, and they must inherit from the built-in Exception class.'''

syntax:

```
class ErrorName(Exception):
   pass
```

Example 1 – Product Availability

```
class ProductNotAvailable(Exception):
   pass
noproducts = int(input('Enter the number of products: '))
if noproducts <= 0:
   raise ProductNotAvailable('Out Of Stock')
else:
   print('Product is Available')
```

Example 2 – Insufficient Balance

```
class InsufficientBalance(Exception):
   pass
balance = 50000
amount = int(input('Enter the withdrawal amount: '))
if amount > balance:
   raise InsufficientBalance('Insufficient Balance')
else:
   print('Withdrawal successful!')
```

Handling User-Defined Exceptions

You can also handle user-defined exceptions using try and except blocks.

Example 1: Product Exception Handling

```
class ProductNotAvailable(Exception):
   pass
def check_product():
   try:
      noproducts = int(input('Enter the number of products: '))
      if noproducts <= 0:
         raise ProductNotAvailable('Out Of Stock')
      else:
         print('Product is Available')
   except ProductNotAvailable as e:
      print('Exception:', e)
      check_product()
check_product()
```

Example 2: Bank Exception Handling

```
class InsufficientBalance(Exception):
   pass
balance = 50000
try:
   amount = int(input('Enter the withdrawal amount: '))
   if amount > balance:
      raise InsufficientBalance('Insufficient Balance')
   else:
      print(f'Withdrawal successful! Remaining balance: {balance - amount}')
```

```
except InsufficientBalance as e:
    print('Exception:', e)
```

Practice Programs (User-Defined Exception Tasks)

1. Write a program to raise an exception if the person's age is below 18 (not eligible for voting).
2. Raise an exception if the person is not eligible for a driving license (age < 18).
3. Raise an exception if the student has not submitted the assignment.
4. Raise an exception if a passport is invalid or expired.

Assertion

Definition:

Assertion is a mechanism to **test a condition** and raise an AssertionError automatically if the condition fails.

It is mainly used for **debugging and validation** during development.

You don't need to mention an exception name explicitly — by default, Python raises an AssertionError.

Syntax: assert condition, 'Error Message'
<---> ----instruction----

Example 1: String Length Validation

```
s1 = input('Enter first string: ')
s2 = input('Enter second string: ')
assert len(s1) == len(s2), 'Length is different'
print(s1 + s2)
```

Example 2: Age Validation

```
age = int(input('Enter your age: '))
assert age >= 18, 'Age must be 18 or older'
print('You are eligible to vote.')
```

we can handle exception by using else,finally but try,except is enough to handle Exception

try–except–else–finally Blocks

Python allows additional blocks else and finally along with try and except for better control.

Example: Player Score Program

```
import random
try:
    target = 100
```

```
        score = random.randint(0, 100)
        res = target / score  # may raise ZeroDivisionError
    except ZeroDivisionError:
        print('Player has not scored anything.')
    else:
        print('Player has scored:', score)
    finally:
        print('Total amount:', score * 1000)
```

Explanation:

- If the score is 0, a ZeroDivisionError occurs and is handled by the except block.
- If no error occurs, the else block executes.
- The finally block executes **in both cases** (used for cleanup or final actions).

==========

Regular Expression

Regular Expression (Regex) in Python

Regular Expression (Regex) is a technique to **define patterns** which help us **search, extract, or validate data** from a large amount of text. It is extremely useful when working with text processing, logs, or data validation.

1. Basic Patterns

1.1 Matching a single character

- [A-Z] → Matches **any one uppercase letter** from A to Z.
  Example: 'A', 'G', 'Z'
- [a-z] → Matches **any one lowercase letter** from a to z.
  Example: 'a', 'g', 'z'
- [0-9] or \d → Matches **any single digit** from 0 to 9.
  Example: '0', '5', '9'

1.2 Matching multiple types of characters

- [A-Za-z] → Matches **any single letter** (uppercase or lowercase)
- [A-Za-z0-9] → Matches **any alphanumeric character**

1.3 Matching multiple occurrences

- {n} → Matches exactly **n occurrences** of the pattern
  Example: [0-9]{3} matches exactly **3 digits**, e.g., '123'
- {m,n} → Matches **between m and n occurrences** of the pattern
  Example: [A-Z]{2,5} matches **2 to 5 uppercase letters**, e.g., 'AB', 'ABCDE'

2. Special Symbols in Regex

Symbol	Meaning	Example
\	Removes special meaning of special characters	\. matches . literally
+	Matches **1 or more** occurrences	[0-9]+ matches 5, 123, 9876
*	Matches **0 or more** occurrences	[a-z]* matches '', 'abc'

? Matches **0 or 1 occurrence** [a-z]? matches '' or 'a'

3. Regex Functions in Python
Step 1: Import the re module

```
import re
```

Step 2: Use findall() and finditer()

1. **re.findall()** → Extracts all occurrences of the pattern from the text

```
res = re.findall('pattern', data)
```

1. **re.finditer()** → Returns an **iterator with match objects** containing index positions

```
out = re.finditer('pattern', data)
```

4. Example: Extracting Phone Numbers

```
s = '''hi to book for any event call us on 9876543210 or email us abc@even.com
you can also reach out on 7898974567 or abc.ka@event.in. We are offering 50% discount for
Diwali i.e 01/12/2025 exactly at 09:00 or 09:30 am. Visit us on www.abcevent.com for pre-booking.
Call us @07:45pm till 12:45am we will be available 05/05/2025.'''
# Pattern to match phone numbers starting with 6-9 and followed by 9 digits
res = re.findall('[6-9][0-9]{9}', s)
out = re.finditer('[6-9][0-9]{9}', s)
print(res)        # Output: ['9876543210', '7898974567']
print(list(out))    # Output: list of match objects with index positions
```

Explanation:

- [6-9] → The first digit of phone number is between 6 to 9
- [0-9]{9} → The next 9 digits can be anything from 0-9

5. Assignment / Practice Regex Patterns

1. **Match Date (formats like 01/12/2025, 05-05-2025, 2025-12-12)**
   Hint: Use [0-9]{2}[-/][0-9]{2}[-/][0-9]{4}
2. **Match Time (formats like 09:00, 07:45pm, 12:45am)**
   Hint: Use [0-9]{2}:[0-9]{2}(?:[ap]m)?
3. **Match Website URL (like www.abcevent.com)**
   Hint: Use www\.[A-Za-z0-9.-]+\.[A-Za-z]{2,}
4. **Match PAN Number (like ABCDE1234F)**
   Hint: Use [A-Z]{5}[0-9]{4}[A-Z]
5. **Match Email ID (like abc@even.com)**
   Hint: Use [a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}

```
s='''i am saran i am from chennai my contact is 123456789012,6428692869,mailid sharan@gmail.com
my ista id is www.instagram.com,www.facebook.com,Kishore,,Abdul Sameer i will join python class at
2:00pm at 4:00pm appointment for to hair style.8737536243, 7653273835'''
import re
res=re.findall('[6-9][0-9]{9}',s)
print(res)


#write an expression to match dates

d='''10/12/2025
09/dec/2025'''

import re
#res=re.findall('[0-9]{2}/[0-9]{2}/[0-9]{4}',d)
res=re.findall('[0-9]{2}/[A-Z a-z]{3}/[0-9]{4}',d)
print(res)


#HH:MM,HH:MM:SS

t='02:00pm at 04:00pm 9:56pm'
res=re.findall('[0-9]{0,2}:[0-9]{2}[a-z]{2}',t)
out=re.finditer('[0-9]{0,2}:[0-9]{2}[a-z]{2}',t)
print(res)
print(list(out)

w='www.facebook.com ,  www.instagram.in'

res=re.findall('www.[A-z a-z]+.[a-z]{2,3}',w)
print(res)


e='abcd12.efg@gmail.in,abcd@gmail.com'

res=re.findall('[A-z a-z 0-9]+.[A-z a-z 0-9]+@[a-z]+.[a-z]{2,3}',e)
print(res)

'''
#Assignment:
#write an expression to match Adhar Number
#write an expression to match PAN Number
#write an expression to match USN Number(ex:01fe20ee456,2SA22CV401....21RU5A0505)
```

```
#write an expression to match Account Number
#write an expression to match emailid
   1.pv99993000@gmail.com
   2.abcd1.efg@event.com
   3.slv.234589@gmail.com
   4.first.last@company.io
   5.ss6788@srmist.edu.in
   6.raj_yahoo@yahoo.com
   7.21ru5a0505@ruce.in
'''
```